Desarrollo de Sistemas de Software
Práctica 2: Catálogo y carrito



Realizado por:

Daniel Alcaide Sánchez

Índice

1. Objetivo y documentación de referencia 2

2. Parte 1: configuración del proyecto 2

3. Parte 2: modelo de datos y repositorio 3

4. Parte 3: operaciones CRUD de productos 4

5. Parte 4: funcionamiento del carrito 4

6. Parte 5: autenticación y control de acceso 6

7. Parte 6: exportación de productos a SQL 7

8. Parte 7: comprobaciones y mejora opcional 7

9. Organización del código y alcance 9

10. Material de referencia 9

1. Objetivo y documentación de referencia

El objetivo de la práctica es desarrollar una aplicación Spring Boot que permita consultar un catálogo de productos, gestionar un carrito de la compra, controlar el acceso a la administración y exportar los productos mediante un script SQL.

Esta memoria relaciona los requisitos del documento *Developing a Shopping Cart with Spring Boot and Thymeleaf* con las clases, métodos y comportamientos implementados. El documento *Arquitectura software de la aplicación* sirve como referencia para la separación en capas. *Spring summary* y *Mi_aplicacion*, incluido en la Carpeta Fundamental, aportan ejemplos de configuración y arranque. El guion identificado como obsoleto se considera un antecedente.

Se utilizaron los HTML proporcionados en la carpeta compartida, así como parte del código de la Carpeta Fundamental, adaptando este material a los requisitos de la práctica.

La implementación amplía el guion con persistencia entre ejecuciones, una cuenta de usuario con permisos limitados y sincronización del carrito con los cambios del catálogo. Se distingue entre las exigencias del guion y estas ampliaciones.

2. Parte 1: configuración del proyecto

Requisito del guion. Crear un proyecto Maven con Spring Boot, incluir las dependencias web, Thymeleaf, JPA, Spring Security y H2, y configurar la conexión a la base de datos, la consola H2 y un usuario predeterminado.

Implementación realizada. El proyecto `Practica1` utiliza Spring Boot 4.1.1 y declara Java 17 en `pom.xml`. Incluye las dependencias de Spring Web MVC, Thymeleaf, Spring Data JPA, Spring Security, H2 y Lombok. También incluye el módulo de consola H2 utilizado por esta versión de Spring Boot.

La clase `Practica1Application`, ubicada en el paquete `com.ejemplo.practica1`, contiene la anotación `@SpringBootApplication` y el método `main`, que inicia la aplicación mediante `SpringApplication.run`.

La configuración se encuentra en `src/main/resources/application.properties`. Los valores principales son:

```
spring.datasource.url=jdbc:h2:file:./data/practica1
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

```
spring.jpa.hibernate.ddl-auto=update
```

El ejemplo del guion emplea una base de datos en memoria. Se utiliza H2 en disco para conservar el catálogo y los carritos de usuarios entre ejecuciones. Las cuentas de la aplicación se definen en `SecurityConfig`, mediante un bean `UserDetailsService`.

2.1. Puesta en marcha

Para ejecutar el proyecto desde Spring Tools se importa la carpeta que contiene `pom.xml` como proyecto Maven y se inicia `Practica1Application` mediante *Run As – Spring Boot App*. Se necesita un JDK compatible, con Java 17 como mínimo, y conexión a Internet para descargar las dependencias que falten.

También se puede ejecutar desde una terminal de Windows, situada en la carpeta `Practica1`, con Java y `JAVA_HOME` configurados:

```
mvnw.cmd spring-boot:run
```

El acceso se realiza desde http://localhost:8080. El Maven Wrapper permite utilizar Maven sin instalarlo por separado.

3. Parte 2: modelo de datos y repositorio

Requisito del guion. Definir una entidad `Product` con identificador, nombre y precio, y una interfaz `ProductRepo` que extienda `JpaRepository`.

Implementación realizada. La entidad `Product` contiene los campos `Long id`, `String name` y `double price`. Se emplean `@Entity`, `@Id` y `@GeneratedValue` con la estrategia `IDENTITY`. Lombok genera el constructor sin argumentos mediante `@NoArgsConstructor`; los getters y setters están escritos en la clase.

Los métodos `equals` y `hashCode` sobrescriben los de `Object` y llevan `@Override`. La igualdad compara el identificador de productos ya persistidos. El código hash se mantiene estable cuando JPA asigna un identificador.

`ProductRepo` extiende `JpaRepository<Product, Long>`. Spring Data JPA proporciona las operaciones de consulta, guardado y eliminación utilizadas por el servicio.

Para representar las cantidades y guardar los carritos se utiliza además la entidad `CartItem`:

@lX@ **Campo Función**

`id` Identificador generado de la entrada del carrito.

`username` Nombre del usuario propietario de la entrada persistida.

`product` Referencia a `Product` mediante una relación `@ManyToOne`.

`quantity` Número de unidades del producto.

`CartItem.getSubtotal()` calcula el precio actual del producto multiplicado por su cantidad. Cada usuario autenticado dispone de un único carrito, representado por el conjunto de entradas asociadas a su nombre.

4. Parte 3: operaciones CRUD de productos

Requisito del guion. Implementar una capa de servicio y un controlador para listar, añadir, editar y eliminar productos.

Implementación realizada. `ProductService` concentra las operaciones sobre el catálogo:

@lX@ **Método**	**Operación realizada**
`getAllProducts()`	Obtiene todos los productos mediante `findAll()`.
`getProductById(id)`	Consulta un producto mediante `findById(id)`.
`saveProduct(product)`	Crea o actualiza un producto mediante `save(product)`.
`deleteProduct(id)`	Elimina las referencias en los carritos guardados y después el producto.

`ProductController` recibe las peticiones y llama al servicio. Las rutas implementadas son:

@llX@ **Método**	**Ruta**	**Operación**
GET	`/products`	Consultar el catálogo.
GET	`/products/new`	Acceder a la creación de un producto.
GET	`/products/edit/{id}`	Consultar el producto que se va a editar.
POST	`/products/add`	Crear un producto.
POST	`/products/edit`	Guardar la edición de un producto.
POST	`/products/delete/{id}`	Eliminar un producto.

El guion propone rutas orientativas. Se mantienen las cuatro operaciones exigidas, usando peticiones GET para consultar y POST para modificar datos. Tras crear, editar o borrar un producto, el controlador redirige a `/admin`.

5. Parte 4: funcionamiento del carrito

Requisito del guion. Crear `CartService` y `CartController` para consultar, añadir y eliminar artículos. El guion admite un carrito simplificado en memoria y recomienda separar los carritos mediante la sesión HTTP.

Implementación realizada. `CartService` utiliza `@SessionScope`. Los visitantes anónimos disponen de una lista temporal por sesión. Para los usuarios autenticados, las entradas se guardan en H2 y se recuperan por nombre de usuario. El método privado `getUsername()` consulta la autenticación y distingue ambas situaciones.

Los métodos principales son:

- `addProduct(product, quantity)`: añade un producto o incrementa su cantidad si ya está en el carrito. No añade cantidades iguales o inferiores a cero.
- `getCartItems()`: obtiene las entradas guardadas del usuario o actualiza la lista temporal del visitante.
- `updateQuantity(productId, quantity)`: cambia las unidades. Si la cantidad es igual o inferior a cero, elimina el artículo.
- `removeProduct(productId)`: elimina el artículo del carrito correspondiente.
- `getTotal()`: suma los subtotales de los artículos actuales.

`CartController` expone GET `/cart` y las operaciones POST `/cart/add/{id}`, `/cart/update/{id}` y `/cart/remove/{id}`. Su método `viewCart()` obtiene los artículos una sola vez y calcula el total sobre esa misma lista.

5.1. Persistencia entre ejecuciones

Al arrancar desde la carpeta del proyecto, H2 almacena los datos en `data/practica1.mv.db`. La ruta es relativa al directorio desde el que se ejecuta la aplicación. Hibernate conserva los datos existentes con `ddl-auto=update`.

Los productos y los carritos de `admin` y `user` se recuperan al reiniciar la aplicación y volver a iniciar sesión. El carrito anónimo pertenece a la sesión y no se guarda en disco. Al iniciar sesión se utiliza el carrito guardado de la cuenta; no se incorpora automáticamente el carrito temporal anterior.

5.2. Cambios de precio y eliminación de productos

Cada entrada mantiene una referencia al producto, en lugar de guardar una copia independiente de su precio. Para usuarios autenticados, `getCartItems()` recupera las entradas junto con sus productos mediante una consulta con `join fetch`. Para anónimos, consulta cada producto mediante `ProductRepo.findById()` y actualiza la referencia.

Por ejemplo, dos unidades de Agua a 0,30 euros suman 0,60 euros. Si el administrador cambia el precio a 0,50 euros, al volver a consultar el carrito esas dos unidades suman 1,00 euro. La actualización se ve al recargar la página o volver a acceder al carrito.

Cuando se borra un producto, `ProductService.deleteProduct()` elimina primero todas las entradas persistidas que lo referencian y después el producto. Ambas operaciones se ejecutan dentro de una transacción. En los carritos anónimos, `getCartItems()` utiliza `removeIf` para retirar los productos que ya no existen. El total se calcula con los artículos restantes.

6. Parte 5: autenticación y control de acceso

Requisito del guion. Permitir consultar públicamente el catálogo, proteger la gestión de productos, reservar `/admin/**` al rol administrador y configurar el inicio y cierre de sesión con Spring Security.

Implementación realizada. `SecurityConfig` utiliza `@Configuration`, `@EnableWebSecurity` y un bean `SecurityFilterChain`. Se definen dos cuentas mediante `InMemoryUserDetailsManager`:

@lllX@ **Usuario**	**Contraseña**	**Rol**	**Permisos**
`admin`	`admin`	ADMIN	Consultar, usar el carrito, gestionar productos y exportar SQL.
`user`	`user`	USER	Consultar el catálogo y utilizar su carrito guardado.
Anónimo	—	—	Consultar el catálogo y utilizar un carrito temporal.

Las contraseñas se codifican con `BCryptPasswordEncoder`. Estas cuentas de demostración se crean al arrancar; los datos persistentes de sus carritos se asocian al nombre de usuario.

GET `/products` y GET `/products/search` son públicos. El resto de las rutas de gestión de productos, así como `/admin/**`, exige el rol ADMIN. Esta restricción amplía la autenticación básica del guion para que la cuenta USER tenga permisos limitados. La comprobación se realiza en el servidor, incluso si se escribe directamente una ruta protegida.

`LoginController` atiende GET `/login`; Spring Security procesa la autenticación enviada a esa misma ruta. Tras acceder correctamente se redirige a `/products`. El cierre de sesión se procesa mediante POST `/logout`, con redirección a `/login?logout`.

6.1. Protección CSRF y consola H2

La protección CSRF permanece activa en las operaciones de la aplicación. Los POST protegidos necesitan la sesión correspondiente y un token válido. La excepción se limita a `/h2-console/**`, para permitir el funcionamiento de la consola. También se configura `frameOptions().sameOrigin()`.

La consola se abre en http://localhost:8080/h2-console/ y utiliza estos parámetros:

```
Driver:    org.h2.Driver
```

```
JDBC URL: jdbc:h2:file:./data/practica1
Usuario: sa
Password: password
```

Su acceso no requiere iniciar sesión como administrador de la aplicación, pero la conexión a H2 utiliza sus propias credenciales. Esta configuración corresponde al entorno local de la práctica; el guion plantea la excepción de H2 para desarrollo.

7. Parte 6: exportación de productos a SQL

Requisito del guion. Crear `DatabaseExportService` y un endpoint en `AdminController` que descargue un script con un INSERT por cada producto almacenado.

Implementación realizada. `DatabaseExportService.exportDatabaseToSql()` consulta todos los productos con `ProductRepo.findAll()` y construye una sentencia por producto, incluyendo identificador, nombre y precio. Por ejemplo:

```
INSERT INTO product (id, name, price) VALUES (1, 'Agua', 0.3);
```

Los apóstrofes de los nombres se escapan duplicándolos y el contenido se codifica en UTF-8. `AdminController.exportDatabase()` devuelve el archivo desde GET `/admin/export`, con la cabecera de descarga `Content-Disposition` y el nombre `productos.sql`. Solo el administrador puede acceder a esta operación.

El archivo exporta los datos de productos exigidos por el guion. Contiene sentencias INSERT para una tabla `product` ya existente; no incluye la creación del esquema ni las entradas de los carritos. Si el catálogo está vacío, el archivo no contiene sentencias INSERT.

8. Parte 7: comprobaciones y mejora opcional

Requisito del guion. Probar las operaciones del catálogo y del carrito, el flujo de autenticación, la protección de la gestión y el contenido del SQL exportado. El guion propone mejoras opcionales como búsqueda de productos o filtrado por precio.

8.1. Prueba automatizada incluida

La clase `Practica1ApplicationTests` contiene `contextLoads()`, que comprueba el arranque del contexto de Spring. Se identifica explícitamente `Practica1Application` como clase de configuración y se utiliza una base de datos H2 en memoria con `create-drop`, separada de los datos reales.

La ejecución de pruebas puede realizarse con:

```
mvnw.cmd test
```

Este test verifica el arranque; las comprobaciones funcionales descritas a continuación se realizaron por separado durante el desarrollo y no forman una suite automatizada incluida en el proyecto.

8.2. Comprobaciones funcionales realizadas

Durante el desarrollo se comprobaron los siguientes comportamientos:

1. Acceso público al catálogo y operaciones de creación, edición y eliminación como administrador.
2. Inicio de sesión con ambas cuentas, rechazo de contraseña incorrecta y cierre de sesión.
3. Bloqueo de la gestión y la exportación para USER y anónimos, incluyendo peticiones directas a rutas protegidas.
4. Incorporación de artículos, suma de cantidades, actualización, eliminación y cálculo del total del carrito.
5. Separación de carritos entre cuentas y entre sesiones anónimas.
6. Conservación del catálogo y de los carritos de usuarios después de detener y volver a arrancar la aplicación.
7. Actualización del precio al consultar el carrito y retirada de productos eliminados del catálogo.
8. Descarga SQL con un INSERT por producto, caracteres acentuados y nombres con apóstrofes.
9. Rechazo de un POST autenticado sin token CSRF y conexión real a la consola H2.

Estas comprobaciones documentan las verificaciones efectuadas durante el desarrollo, no una nueva ejecución de pruebas al redactar esta memoria.

8.3. Búsqueda por nombre

Se implementó la mejora opcional de búsqueda. GET `/products/search` recibe el parámetro `name`. `ProductController.searchProducts()` elimina los espacios al principio y al final, y llama a `ProductService.searchProducts()`.

El servicio utiliza `ProductRepo.findByNameContainingIgnoreCase(name)`. La consulta busca coincidencias parciales sin distinguir mayúsculas de minúsculas: buscar *agua* permite encontrar *Agua mineral*. Una búsqueda vacía muestra todos los productos; si no hay coincidencias, se comunica que no se encontraron resultados. La búsqueda es pública y permite añadir al carrito los productos encontrados.

9. Organización del código y alcance

La organización sigue las capas representadas en el PDF de arquitectura:

@lX@ **Paquete**	**Responsabilidad**
`controller`	Recibir peticiones y llamar a los servicios.
`service`	Gestionar catálogo, carrito y generación del SQL.
`repository`	Proporcionar las operaciones de acceso a productos con Spring Data JPA.
`model`	Representar productos y entradas del carrito.
`config`	Definir cuentas y reglas de autenticación y autorización.

Los servicios y controladores emplean `@Autowired` para recibir dependencias gestionadas por Spring. `@PersistenceContext` proporciona el `EntityManager` utilizado para persistir las entradas del carrito y ejecutar consultas. `@Transactional` agrupa las modificaciones relacionadas, como la eliminación de un producto y sus entradas guardadas.

9.1. Correspondencia con los documentos de apoyo

Spring summary explica anotaciones y contiene ejercicios independientes con entidades `Todo` y servicios REST. Se utiliza como apoyo conceptual para la inyección de dependencias y JPA; esos ejercicios no forman parte de la implementación del carrito descrita por el guion principal.

Mi_aplicacion propone una clase principal y una segunda clase `ServletInitializer`, que extiende `SpringBootServletInitializer`. En el estado del código documentado está presente la clase principal con `main`; no figura la segunda clase. El proyecto conserva el empaquetado JAR y el arranque con servidor integrado. Por tanto, el despliegue tradicional como WAR y la estructura de dos clases indicada en ese documento no se presentan como implementados.

10. Material de referencia

- *Developing a Shopping Cart with Spring Boot and Thymeleaf*: requisitos principales, partes 1 a 7.

- *Arquitectura software de la aplicación*: organización de controladores, servicios, repositorios, seguridad y exportación.
- *Spring summary*: anotaciones, inyección de dependencias y persistencia JPA.
- *Mi_aplicacion*: ejemplo de clases de arranque incluido en la Carpeta Fundamental.
- Carpeta compartida y Carpeta Fundamental: material de partida reutilizado en el proyecto.

Repositorio del proyecto: https://github.com/Wid4er/DDS-Practica-1.